Technical Exercise - Findings and Fixes

I first ran the frontend and backend and tested the API and UI manually. I focused on issues that affected correctness, reliability and user experience.

1. SQL filtering bug
   In `TaskRepository.java`, the search query had an AND/OR precedence problem. Because the conditions were not grouped, archived tasks could appear in search results and the status filter did not always apply correctly. I reproduced this through API requests such as `q=api` and `status=OPEN`. I added parentheses around the search conditions and applied the same correction to the SQL reference files.

2. Frontend request state
   In `useTasks.js`, a failed request could leave the page stuck on the loading state. I tested this by stopping the backend. I fixed the loading/error lifecycle and added request cancellation so an older request cannot overwrite a newer search result.

3. Pagination and API validation
   Changing the search or status filter did not reset the page, which could result in an empty page. I reset the page to 1 whenever filters change. I also fixed invalid `status`, `page` and `pageSize` values so they return HTTP 400 instead of 500.

4. Artificial latency
   The controller intentionally added a delay to requests. This made the application feel slow without serving any real purpose, so I removed it and verified the response time improved.

5. Search correctness and testing
   `%` and `_` were being treated as SQL wildcards. I escaped them so user input is treated literally. I also added a focused repository test covering archived filtering, status filtering and combined search. I verified that the test fails when the original SQL bug is reintroduced.

6. UX and accessibility
   I improved loading, error and empty states, added retry and clear-filter actions, improved keyboard accessibility and focus behavior, and added result counts and live status information.

7. UI and responsive improvements
   I refined the visual theme and controls, removed the separate priority column, made

priority a compact indicator, kept assignees simple, and created a mobile task-card view so users do not have to horizontally scroll a large table.

I intentionally did not redesign the whole architecture. Database-side pagination, Oracle redesign, DTO/service layers and other larger changes were left out because the exercise asked for a focused patch and limited time.

The main goal throughout was to fix the highest-value issues while keeping the changes small, understandable and testable.